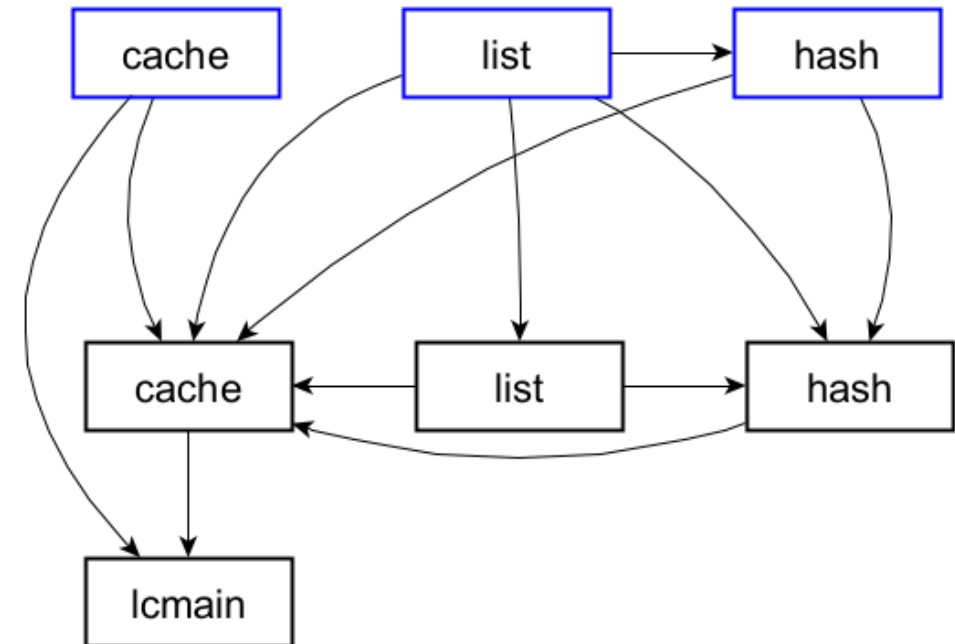
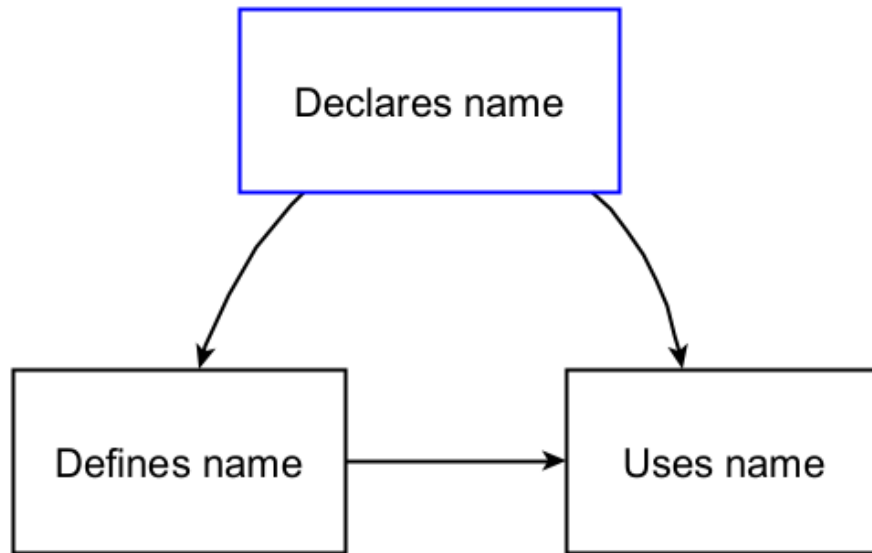


СИСТЕМЫ СБОРКИ

Мотивация, основные решения, подводные камни.

К. Владимиров, Yadro, 2024
mail-to: konstantin.vladimirov@gmail.com

Зависимости между файлами



Include what you use

- Циклические зависимости между хедерами страшнее чем между модулями, поэтому минимизируйте включения в хедер.
- В данном случае можно детектировать лишние хедера везде по проекту.

```
$ iwyu ./source/hash.c -I./include
```

source/hash.c should add these lines:

```
#include <stdlib.h>    // for calloc, free
```

source/hash.c should remove these lines:

```
- #include <malloc.h>  // lines 13-13
```

```
- #include <math.h>    // lines 14-14
```

Простейший способ сборки

```
if [ -z ${FLAGS+x} ]  
then  
    echo "FLAGS unset, default to -O2"  
    FLAGS="-O2"  
fi  
  
gcc $FLAGS -c -I./include LCmain.c -o LCmain.o  
gcc $FLAGS -c -I./include ./source/cache.c -o cache.o  
gcc $FLAGS -c -I./include ./source/hash.c -o hash.o  
gcc $FLAGS -c -I./include ./source/list.c -o list.o  
gcc $FLAGS LCmain.o cache.o hash.o list.o -o LCsep.x
```

- Покритикуйте это решение.

Декларативная модель

- Вы описываете **зависимости**, а не процесс.

```
FLAGS ?= -O2
```

```
LCmain.o: LCmain.c
```

```
→ gcc $(FLAGS) -c LCmain.c -o LCmain.o
```

- Стрелочкой " → " я обозначаю символ табуляции (обязательный).
- Переменная FLAGS может быть установлена извне или изнутри.

```
make FLAGS="-g -O0"
```

- Операция ?= означает "установить если не определена по правилам shell".

```
make FLAGS=
```

Общее правило

- Любое правило (рецепт) в make записывается единообразно.

```
<таргет>: [<реквизиты>]  
→ [команды shell]
```

- Таргет обычно означает файл и make следит за датой обновления файла.
- Рецепт пересобирается только если изменился хотя бы один реквизит либо если реквизита вообще нет.

```
clean:  
→ rm -rf *.x *.o *.log
```

Простейший makefile

```
FLAGS ?= -O2
```

```
TESTS = ./Tests
```

```
all: LCmain.o cache.o hash.o list.o
```

```
→ gcc $(FLAGS) -I./include LCmain.o cache.o hash.o list.o -o LC.x
```

```
LCmain.o: LCmain.c
```

```
→ gcc $(FLAGS) -I./include -c LCmain.c
```

```
cache.o: ./source/cache.c
```

```
→ gcc $(FLAGS) -I./include -c ./source/cache.c -o cache.o
```

```
clean:
```

```
→ rm -rf *.x *.o *.log
```

Заглушить вывод строки

- По умолчанию make показывает все строки исполнения.
- Если нам надо заглушить вывод, мы указываем @ впереди.

testrun: LC.x

```
→ @for i in $(TESTS)/*.dat; do \  
→   echo "$$(basename ${i})"; \  
→   ./LC.x < ${i}; \  
→   echo ""; \  
→ done > all.log  
→ @if diff -w all.log $(TESTS)/corr.log; then \  
→   echo "Tests pass"; else echo "Tests failed"; \  
→ fi
```

Проблема простого примера

- Компилятор прибит гвоздями.

LCmain.o: LCmain.c

→ `gcc $(FLAGS) -I./include -c LCmain.c -o LCmain.o`

- Для компиляторов, линкеров и т.д. в make есть стандартные переменные.

LCmain.o: LCmain.c

→ `$(CC) $(CFLAGS) -c LCmain.c -o LCmain.o`

- Но прежде чем продолжать, давайте разберёмся с переменными.

Ленивые переменные

```
$ cat lazyassgn.mak
```

```
baz = Nobody
```

```
bar = Hello $(baz)
```

```
baz = World
```

```
.PHONY: all
```

```
all:
```

```
→ @echo "bar = $(bar)"
```

```
$ make -f lazyassgn.mak
```

```
bar = Hello World
```

PHONY таргеты это специальные таргеты, которые не соответствуют никаким результирующим файлам.

Таргеты вида all, install, clean, info, distclean, check рекомендуется всегда такими делать, чтобы случайный файл с таким именем в папке не помешал сборке.

Не надо бороться с ленивостью

- Пример плохого кода который заходит в большое количество подпапок.

```
SUBDIRS = foo bar baz
```

```
.PHONY: subdirs
```

```
subdirs:
```

```
→ for dir in $(SUBDIRS); do \  
→   $(MAKE) -C $$dir; \  
→ done
```

- Вы потеряли параллельность.
- Ошибки в рекурсивной сборке будут проигнорированы. Можно вставить логику их обнаружения, но тогда она будет работать при вызове make -k, когда мы как раз **хотим** проигнорировать ошибки.

Ленивые списки

- Исправим ситуацию.

```
SUBDIRS = sub1 sub2 sub3
```

```
.PHONY: subdirs
```

```
subdirs: $(SUBDIRS)
```

```
.PHONY: $(SUBDIRS)
```

```
$(SUBDIRS):
```

```
→ @$(MAKE) -C $@
```

Список в make задаётся перечнем с использованием пробелов. Список с одной стороны является как и всё просто строчкой.

Но список может быть раскрыт, причём если он раскрыт слева в рецепте, он становится несколькими таргетами с тем же рецептом.

- Обратите внимание на автоматическую переменную `$@`.

Автоматические переменные

LCmain.o: LCmain.c

→ \$(CC) \$(CFLAGS) -c LCmain.c -o LCmain.o

LCmain.o: LCmain.c

→ \$(CC) \$(CFLAGS) -c \$^ -o \$@

\$@ имя таргета.

\$^ имена всех реквизитов.

\$< имя первого реквизита.

- Также возможно отдельно оперировать с частями имен относящимся к папкам и файлам: \$(@D), \$(@F) и т.д.

Ленивая бесконечность

```
bar = Hello $(baz)  
baz = World
```

```
.PHONY: all
```

```
all:
```

```
→ @echo "bar = $(bar)"
```

- Неинициализированная ленивая переменная откладывается в разрешении.
- Но как вы думаете, а что если ошибочно написать:

```
bar = Hello $(bar)
```

- Получим ли мы бесконечный вывод?

Энергичные присвоения

- Можно зафорсить вычисления.

```
bar := Hello $(baz)
baz := World
```

```
.PHONY: all
all:
→ @echo "bar := $(bar)"
```

- На экране просто Hello.
- Неинициализированная энергичная переменная это пустая строка.
- Но ни ленивые ни энергичные присваивания не сработают на внешнюю переменную.

Override

- Чтобы записать, дописать или переписать переменную, заданную снаружи, надо указать `override`.

```
CFLAGS ?= -O2
```

```
COMMONINC = -I./include
```

```
override CFLAGS += $(COMMONINC)
```

- Теперь будет работать:

```
make CFLAGS="-g -O0"
```

Промежуточные уроки

- Используйте стандартные переменные для компиляторов, линкеров и т.д.
- Используйте также стандартные переменные для флагов.
- Помечайте PHONY те таргеты, которые не соответствуют файлам.
- Старайтесь не бороться с ленивостью, а использовать её.
- Используйте override если вы предполагаете что переменная задаётся извне.
- Не пишите сложные shell-скрипты внутри makefiles.
- Используйте автоматические переменные.

Новый подход, новые проблемы

```
CC ?= gcc
CFLAGS ?= -O2
COMMONINC = -I./include

override CFLAGS += $(COMMONINC)

LCmain.o: LCmain.c
→ $(CC) $(CFLAGS) -c $^ -o $@

cache.o: cache.c
→ $(CC) $(CFLAGS) -c $^ -o $@

list.o: list.c
→ $(CC) $(CFLAGS) -c $^ -o $@
```

- Много абсолютно однотипных строчек.
 - СС в действительности не редефайнит умолчание
- ```
$ make -f makefile.v1.mak
cc -O2 -I./include
 -c LCmain.c -o LCmain.o
```
- Тогда как вроде бы мы выставили явно gcc.

# Проверка на default

- Есть переменные (ARFLAGS, ASFLAGS, CFLAGS, ...) у которых default undefined.
- Есть переменные (AR, AS, CC, ...) у которых есть default values.

```
ifeq ($(origin CC),default)
 CC = gcc
endif
```

- Функция origin интересна тем, что рассказывает откуда пришла переменная. Её возможные значения: undefined, default, environment, file и прочие.
- К слову мы ещё не говорили о функциях.

# Функции в make

- Функция вызывается как `$(function [<arguments>])`.

```
cfiles = main1.c foo.c main2.c bar.c
```

```
mains = main1.c main2.c
```

```
filtered = $(filter-out $(mains),$(cfiles))
```

- Из всех функций главной для нас является `patsubst`. Как аргументы она берёт `pattern`, `replacement`, `text`.

```
objs = $(patsubst %.c, %.o, $(cfiles))
```

- У неё есть второй вариант `$(var:pattern=replacement)`.

```
ofilt = $(filtered:%.c=%.o)
```

# Pattern rules

- Чтобы разобраться с однотипными рецептами, можно написать общий.

```
CSRC = LCmain.c source/cache.c source/hash.c source/list.c
COBJ = $(CSRC:%.c=%.o)
```

```
%.o : %.c
```

```
→ $(CC) $(CFLAGS) -c $^ -o $@
```

```
LC.x: $(COBJ)
```

```
→ $(CC) $^ -o $@ $(LDFLAGS)
```

- На самом деле такой рецепт не надо даже писать т.к. он уже есть.
- Но в данном случае будет проблем: объектные файлы начнут возникать внутри папок.

# Пишем свои pattern rules

```
CSRC = LCmain.c source/cache.c source/hash.c source/list.c
COBJ = LCmain.o cache.o hash.o list.o
```

```
%.o : source/%.c
→ $(CC) $(CFLAGS) -c $^ -o $@
```

```
LC.x: $(COBJ)
→ $(CC) $^ -o $@ $(LDFLAGS)
```

- Правило для LCmain взято как implicit.
- Тут я хотел бы остановиться на одном неприятном моменте.

# Wildcards

- Многие любят делать вот так.

```
CSRC = LCmain.c $(wildcard source/*.c)
```

- Сами по себе wildcards ничем не плохи, но тут у меня есть концептуальное возражение.
- Мне не очень нравится система сборки, которая считает своими таргетами весь мусор который она найдёт в папке source.
- Опыт показывает, что списки файлов лучше прибавать намертво простым перечислением.

# ... и ещё одно небольшое животное.

- Пока всё хорошо, но мы, кажется, пропустили слона в комнате.

`%.o : source/%.c`

`→ $(CC) $(CFLAGS) -c $^ -o $@`

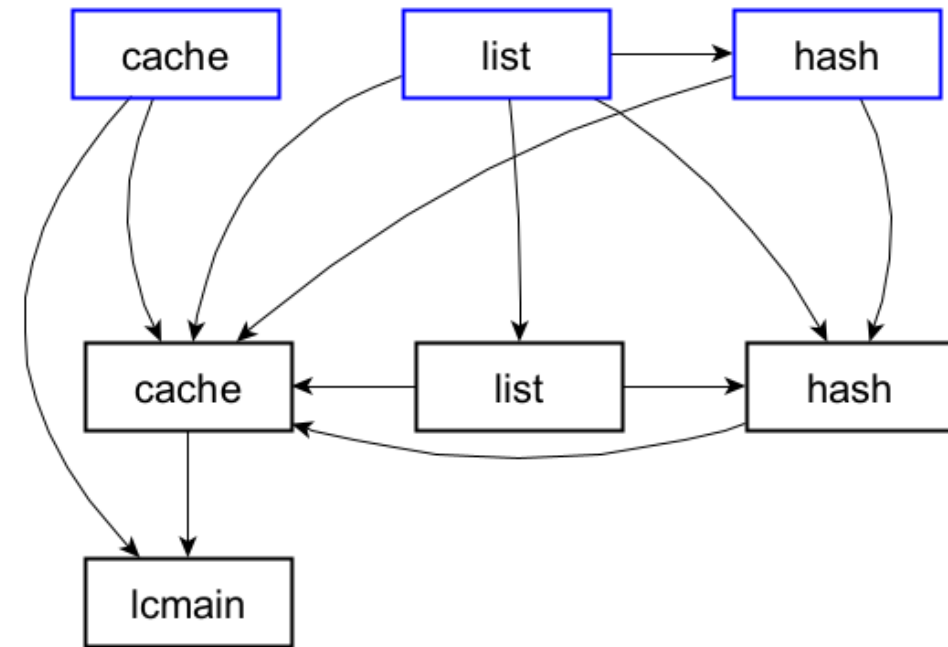
`.PHONY: all`

`all: LC.x`

`LC.x: $(COBJ)`

`→ $(CC) $^ -o $@ $(LDFLAGS)`

- Кто-нибудь его заметит?



# Зависимости от хедеров

- Сейчас у нас make не перестраивает проект если изменился хедер.
- На пути к этому исправлению мы должны выучить новые опции компилятора.

```
$ gcc -I./include -E source/cache.c -MM -MT cache.o
```

```
cache.o: source/cache.c include/cache.h include/hash.h
include/list.h
```

- После MT мы указываем make target и компилятор сам собирает для нас зависимости. Это становится бесконечно сложнее для C++ modules но к счастью мы не о них.
- Разумеется лучше всё это упаковать в implicit rules.

# Включение файлов

- Директива `include` приостанавливает чтение текущего `makefile` и включает в него всё что указано в списке `include` файлов.

```
DEPS = $(COBJ:.o=.d)
```

```
%.o : source/%.c
```

```
→ $(CC) $(CFLAGS) -c $< -o $@
```

```
%.d : source/%.c
```

```
→ $(CC) -E $(CFLAGS) $< -MM -MT $(@:.d=.o) > $@
```

```
include $(DEPS)
```

- Думайте об этом так: первое считывание `makefile` открывает все инклюды, после чего второе считывание собирает все нужные зависимости.

# Убираем слишком общие правила

- Хороший тон это использовать для ваших переменных static pattern rules

```
objects = foo.o bar.o
```

```
all: $(objects)
```

```
$(objects): %.o: %.c
```

```
→ $(CC) -c $(CFLAGS) $< -o $@
```

- Так мы не претендуем на то что мы определяем все такие правила, а только определяем их для списка наших целей.

# Условное включение

- Чтобы инклюды не пересобирались на make clean можно отфильтровать имена такого рода таргетов при включении.

```
targets which we have no need to rebuild
NODEPS = clean
```

```
ifeq (0, $(words $(findstring $(MAKECMDGOALS), $(NODEPS))))
include $(DEPS)
endif
```

- Также есть синтаксис "-include" когда мы игнорируем отсутствующие файлы.

# Почти идеально

- Мы близки к совершенству. Можно ещё улучшить тесты, чтобы их можно было запускать в параллель. Тут как раз wildcard уместен.

```
TESTFILES=$(wildcard $(TESTS)/*.dat)
```

```
.PHONY: testrun
testrun: $(TESTFILES)
```

```
.PHONY: $(TESTFILES)
$(TESTFILES): $(OUT_O_DIR)/LC.x
→ @$(ROOT_DIR)/runtest.sh $@ $(OUT_O_DIR)/LC.x
```

- Какие у нас ощущения от нашего совершенства?

# Промежуточные уроки

- Помните, что у некоторых переменных есть default values и проверяйте их правильно.
- Используйте pattern rules для уменьшения бойлерплейта.
- Выучите список implicit rules (по ссылке внизу этого слайда).
- Используйте static pattern rules чтобы не конфликтовать с неявными правилами.
- Не злоупотребляйте использованием wildcards, используйте их там, где не может быть зависимостей по сборке, например для тестов.
- Не забывайте про зависимости от хедеров.

# Обсуждение

- А давайте всё это выкинем? 😄

# Предназначение cmake

- CMake это собственно не система сборки.
- Это генератор файлов для систем сборки, например для make, но не только.

```
cmake -G "Unix Makefiles"
```

```
cmake -G "Ninja"
```

```
cmake -G "Visual Studio 17 2022"
```

- Из одного и того же описания cmake вы получаете файлы сборки для многих разных бэкендов.
- То есть это вещь гораздо более высокоуровневая.

# Проект для problem LC

```
cmake_minimum_required(VERSION 3.11)
project("problem_lc")
```

```
SET(SRCS source/list.c source/cache.c source/hash.c)
```

```
add_library(lclib STATIC ${SRCS})
target_include_directories(lclib PUBLIC "include")
```

```
add_executable(LC.x LCmain.c)
target_link_libraries(LC.x lclib)
```

- Это всё что нужно с точки зрения описания проекта (и мы бесплатно получаем кучу интересных вещей).

# Сборка проекта

- Теперь для конфигурации сборки мы можем указать желаемую папку и это будет подхвачено без необходимости писать поддержку.

- Первый шаг: конфигурация сборки.

```
$ cmake -DCMAKE_BUILD_TYPE=Release -S . -B build
```

- Второй шаг: собственно сборка. Можно сделать и без cmake, но хороший тон использовать cmake.

```
$ VERBOSE=1 cmake --build ./build
```

- Можно убедиться, что на этапе конфигурации мы сконфигурировали зависимости правильно.

# Добавляем тесты

- Нет ничего проще.

```
SET(TESTS ${CMAKE_SOURCE_DIR}/Tests)
file(GLOB testfiles "Tests/*.dat")
foreach(file ${testfiles})
 add_test(NAME ${file}
 COMMAND bash -c "${CMAKE_SOURCE_DIR}/runtest.sh ${file} ./LC.x"
 WORKING_DIRECTORY ${CMAKE_CURRENT_BINARY_DIR})
 set_tests_properties(${file} PROPERTIES DEPENDS LC.x)
endforeach()
```

- Заметим интересное возвращение императивности.

# Отвязанность от shell

- Переменные не являются shell-переменными как в make. Чтобы получить shell-настройку вам надо лезть в env, но обычно это не нужно.
- Для вывода сообщений используется message, а не shell.

```
message(STATUS "MYVAR = ${MYVAR}")
```

- Вместо установки и использования  $\$(CC)$  есть переменная для конфигурации.

```
$ cmake -DCMAKE_C_COMPILER=clang
```

- Если она не подана, то переменная CMAKE\_C\_COMPILER берёт своё значение из  $\$(CC)$ . Аналогично работает переменная CMAKE\_C\_FLAGS.

# Обсуждение

- Как ни парадоксально, но `stake` куда императивнее и значит куда проще `make`.
- Дело в том, что в нём вы не совсем добавляете зависимости, а добавляете код для добавления зависимостей, а он вполне императивен.
- Но `stake` мало того, что императивен. Он ещё и объектно-ориентирован.

# Индивидуальная работа с таргетами

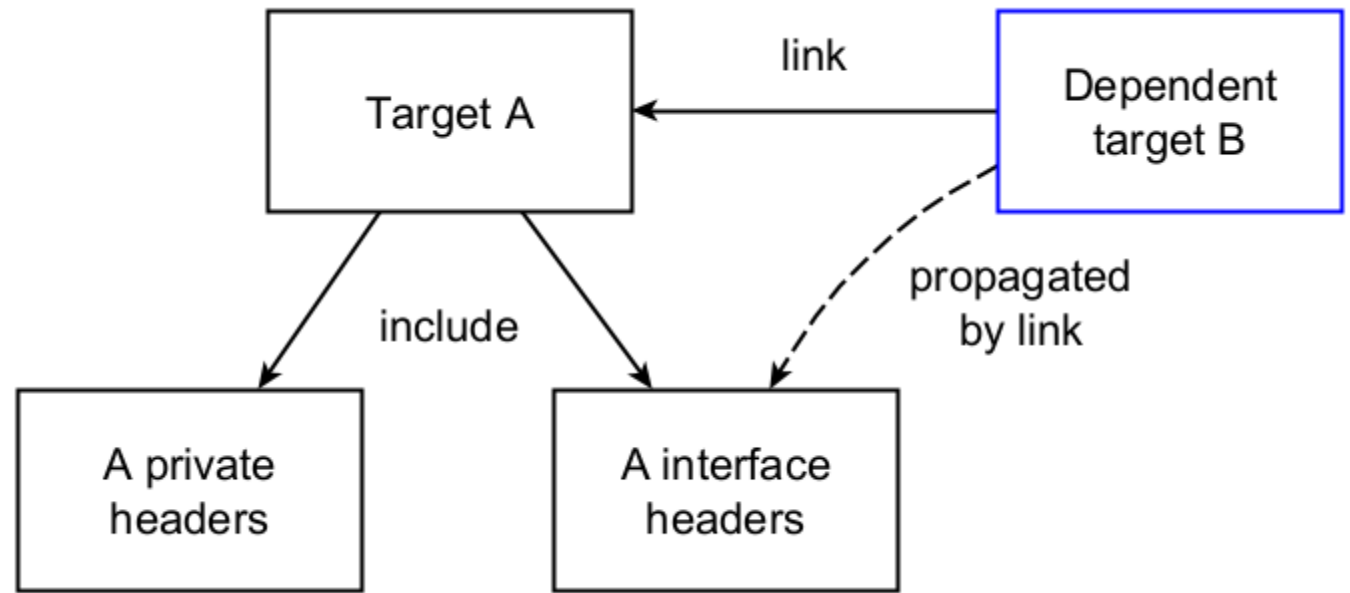
- В stake нормально работать с целями индивидуально, а не ставить флаги компиляции сразу на все.
- Даже если вы работаете с каким-то списком целей одинаково.

```
set(APPS allbraces;allperms)
```

```
foreach(APP ${APPS})
 add_executable(${APP} ${APP}.cc)
 target_compile_features(${APP} PRIVATE cxx_std_20)
 target_link_libraries(${APP} frame)
 target_link_libraries(${APP} comblib)
endforeach()
```

# Интерфейсные библиотеки

- Если библиотека состоит только из хедеров, в ней нечего линковать и её можно добавить как интерфейсную библиотеку
- Тогда "линковка" с ней будет подхватывать её публичные пути для включения.
- Это общий механизм. Любая линковка пропагирует интерфейсные зависимости.



```
add_library(frame INTERFACE)
target_include_directories(frame INTERFACE framework)
```

# Состав таргета

- `target_link_libraries` – любая в том числе транзитивная зависимость по сборке.
- `target_compile_features` – некоторые опции, такие как стандарт, не зависящие от синтаксиса опций конкретного компилятора.
- `target_compile_definitions` – дефайны (в том числе экспортируемые).
- `target_include_directories`, `target_link_directories` – пути (в том числе экспортируемые) для компиляции и линковки. Старайтесь их не указывать явно, а получать из интерфейса библиотек.
- `target_compile_options`, `target_link_options` – прямое указание опций. Старайтесь использовать `features` где возможно.

# Выражения генератора

- Вы можете в момент конфигурации знать что вы конфигурируете и, например, передавать в таргет дефайны, зависящие от этого.

```
target_compile_definitions(MyTarget PRIVATE
 $$<CONFIG:Debug>:DEBUG_ONLY=1>
 $$<CONFIG:Release>:RELEASE_ONLY=1>
 FOO=3
)
```

- Это выглядит пугающе, но на самом деле довольно просто.

```
target_compile_options(myapp
 PRIVATE $$<COMPILE_LANGUAGE:CXX>:-fno-exceptions>
)
```

# Иерархия поддиректорий

- Считается хорошим тоном иметь в проекте stake, делегирующий к вложенным директориям.

```
add_library(frame INTERFACE)
```

```
target_include_directories(frame INTERFACE framework)
```

```
add_library(comblib INTERFACE)
```

```
target_include_directories(comblib INTERFACE combinatorial)
```

```
add_subdirectory(apps)
```

```
add_subdirectory(tests)
```

- Так мы избегаем убер-сборки всего в одном месте и разделяем сборку по "функциям".

# Работа со scope

- Любая переменная доступна через все add\_subdirectory вниз.
- Можно установить переменную в parent\_scope, но лучше этого избегать.

```
concrete/apps запомнили папку до приложения
set(${APP}_app ${CMAKE_BINARY_DIR}/apps/${APP} PARENT_SCOPE)
```

```
concrete/tests использовали её.
set(APP_VAR ${${APP}_app})
```

- Это оправдано в случае когда мы выносим разнородные тесты в общую папку.
- Всегда задавайтесь вопросом не ошибка ли это в проектировании.

# Окончательные уроки

- Ваша система сборки это код и относитесь к ней как к коду.
- Старайтесь не трогать глобальное состояние. Работайте индивидуально с таргетами. Используйте `target_link_libraries` для зависимостей.
- Тем более не лезьте в стандартные переменные. Оверрайд `CMAKE_C_FLAGS` не заканчивается ничем хорошим.
- Все переменные итеративные и не равны переменным `shell`. Списки разделены через ";" или переносом строки.
- Большая часть того, что вы хотите сделать уже поддерживана из коробки. Хороший `stake` максимально это переиспользует.

# Больше про cmake

- Daniel Pfeifer – Effective CMake, C++Now'2017
- Mathieu Ropert – Using Modern CMake Patterns to Enforce a Good Modular Design, CppCon'2017
- Craig Scott – Deep CMake for Library Authors, CppCon'2019
- CB Bailey – Things I Learnt While Trying to Avoid Becoming a CMake Expert, ACCU'2022
- Hans Vredeveld – CMake: A Case Study, ACCU'2023